

OpenDerisk: An Industrial Framework for AI-Driven SRE, with Design, Implementation, and Case Studies

Peng Di*, Faqiang Chen*, Xiao Bai, Hongjun Yang, Qingfeng Li, Ganglin Wei, Jian Mou, Feng Shi, Keting Chen, Peng Tang, Zhitao Shen, Zheng Li, Wenhui Shi, Junwei Guo, and Hang Yu

Ant Group, China

<https://github.com/derisk-ai/OpenDerisk>

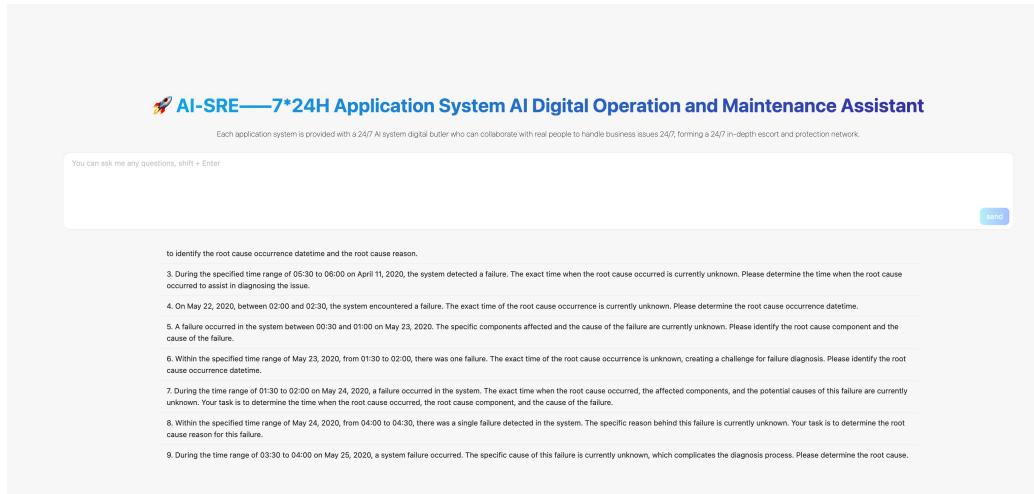


Figure 1: The web-based user interface for OpenDerisk is open source and available at <https://github.com/derisk-ai/OpenDerisk/>

Abstract

The escalating complexity of modern software imposes an unsustainable operational burden on Site Reliability Engineering (SRE) teams, demanding AI-driven automation that can emulate expert diagnostic reasoning. Existing solutions, from traditional AI methods to general-purpose multi-agent systems, fall short: they either lack deep causal reasoning or are not tailored for the specialized, investigative workflows unique to SRE. To address this gap, we present OpenDerisk, a specialized, open-source multi-agent framework architected for SRE. OpenDerisk integrates a diagnostic-native collaboration model, a pluggable reasoning engine, a knowledge engine, and a standardized protocol (MCP) to enable specialist agents to collectively solve complex, multi-domain problems. Our comprehensive evaluation demonstrates that OpenDerisk significantly outperforms state-of-the-art baselines in both accuracy and efficiency. This effectiveness is validated by its large-scale production deployment at Ant Group, where it serves over 3,000 daily users across diverse scenarios, confirming its industrial-grade scalability and practical impact. OpenDerisk is open source and available at <https://github.com/derisk-ai/OpenDerisk/>

*Equal Contribution. Correspondence to: Peng Di <dipeng.dp@antgroup.com> and Faqiang Chen <faqiang.cf@antgroup.com>

1 Introduction

The complexity of modern software—defined by distributed microservices, cloud-native architectures, and relentless release cycles—has surpassed human cognitive scale. For Site Reliability Engineering (SRE) teams, this creates an unsustainable operational burden, making AI-driven automation an operational necessity, not a luxury [BASS et al. \(2025\)](#). However, the central challenge is profound: SRE tasks are not simple computations but complex cognitive investigations. **Whether performing Root Cause Analysis (RCA) or assessing risk, an engineer must synthesize disparate signals into a coherent causal narrative through hypothesis testing and deep system knowledge.** This sophisticated, diagnostic reasoning has eluded previous automation efforts focused on mere pattern matching. This impasse raises a pivotal question: **can we architect a system that emulates the investigative sense-making of an expert SRE?**

The pursuit of SRE automation has produced generations of tools, each with inherent limitations [Bass et al. \(2015\)](#). Traditional learning-based methods, such as causal discovery [Chakraborty et al. \(2023\)](#); [Bi et al. \(2024\)](#); [Arnold et al. \(2007\)](#); [Li et al. \(2022a\)](#) and dependency graph analysis [Zheng et al. \(2024\)](#); [Wang et al. \(2023a\)](#), are often constrained by the immense complexity of real-world systems. More recently, multi-agent frameworks have emerged. The state-of-the-art OpenRCA [Xu et al. \(2025\)](#) uses a multi-agent system for RCA, but its generalist problem-solving model struggles to orchestrate deep, multi-domain expertise and is difficult to extend. Similarly, general-purpose systems, like MetaGPT, TaskWeaver and others [Hong et al. \(2024\)](#); [Zhang et al. \(2024a\)](#); [Qiao et al. \(2024\)](#), are fundamentally tailored for *generative* development, not the investigative and diagnostic workflows central to SRE.

To bridge these gaps, we present OpenDerisk, a specialized multi-agent framework designed to augment, not just automate, human expertise. Its architecture is built on four integrated core components: a **Multi-Agent System** that enables adaptive collaboration patterns; a pluggable **Reasoning Engine** that endows each agent with multi-step cognitive abilities; a sophisticated **Knowledge Engine** that grounds their analysis in domain-specific data; and a standardized **Model Context Protocol (MCP)** that ensures modularity and extensibility.

We validate this architecture through a comprehensive evaluation focused on three key research questions. We first demonstrate that OpenDerisk’s multi-agent approach **significantly outperforms monolithic agents in both accuracy and efficiency (RQ1)**. We then assess its **adaptability by successfully integrating new knowledge and swapping foundational LLMs (RQ2)**. Finally, an in-depth ablation study quantifies the **crucial contribution of each architectural component (RQ3)** to the system’s overall performance.

This proven effectiveness, combined with its inherent flexibility, allows OpenDerisk to function as a practical and powerful “co-pilot” for SRE teams. This is substantiated by its successful production deployment at Ant Group, where, in just three months, it was adopted for **13 new application scenarios** with over **50 new specialized agents** created by developers. Today, the platform serves more than **3,000 daily users** and executes over **60,000 runs per day**, demonstrating its industrial-grade impact and scalability.

This paper’s primary contributions are as follows:

- **Design:** We present a novel framework architecture for SRE where specialist agents, each equipped with a pluggable reasoning engine and grounded in expert knowledge, collaborate within a diagnostic-native paradigm that is governed by a protocol-driven (MCP) approach.
- **Implementation:** We detail the implementation of OpenDerisk as a robust, industrial-grade system. This includes key architectural optimizations such as *advanced context engineering* to manage long-running tasks, *sophisticated agent orchestration* for dynamic collaboration, and integrated *visualization and human-in-the-loop features* to enhance user trust and control.
- **Large-Scale Empirical Validation:** We provide extensive evidence of OpenDerisk’s effectiveness and scalability. This validation includes quantitative metrics from its successful production deployment at Ant Group, where it serves over **3,000 daily users** and executes **60,000+ runs**

per day, as well as qualitative insights from in-depth industrial case studies on diverse SRE workflows.

- **Open-Source Contribution:** As a key contribution, we have open-sourced this core framework¹ to foster research and reproducibility.

2 Related Work

Our work is positioned at the intersection of SRE automation, LLM-based software engineering, multi-agent systems, and context engineering. This section reviews the state of the art in each domain and situates the contributions of OpenDerisk.

SRE Automation. The pursuit of automated SRE, often under the umbrella of AIOps, has a rich history focused on mitigating the operational burden of complex systems. Early efforts centered on statistical methods for anomaly detection [Chandola et al. \(2009\)](#) and log analysis [He et al. \(2016\)](#); [Aué et al. \(2018\)](#); [Cândido et al. \(2021\)](#) to identify deviations from normal behavior. More advanced approaches utilize graph-based structures to model system dependencies and pinpoint root causes. Frameworks like CloudRCA [Li et al. \(2022b\)](#) and MicroRCA [Wang et al. \(2023b\)](#) leverage service dependency graphs and call chains to trace the propagation of failures. RCAEval [Pham et al. \(2025; 2024a,b\)](#) offers an open-source benchmark with an evaluation framework for root cause analysis (RCA) in microservice systems. Concurrently, methods based on causal discovery [Spirtes et al. \(2000\)](#) attempt to infer the underlying causal graph from observational data to distinguish correlation from causation [Xin et al. \(2023\)](#). While powerful at correlating symptoms across vast datasets, these AI methods fundamentally lack deep, semantic reasoning. They can identify anomalous patterns and likely failure paths but struggle to interpret the why behind an incident, a cognitive leap that requires understanding system logic, documentation, and operational context. OpenDerisk bridges this gap by integrating the statistical prowess of these methods with the semantic and causal reasoning capabilities of Large Language Models (LLMs).

LLM-based Software Engineering. The advent of LLMs has revolutionized the software development lifecycle (SDLC). Foundational models like Codex [Chen et al. \(2021\)](#) demonstrated remarkable capabilities in code generation, leading to a new generation of AI-powered software engineering agents. Systems like SWE-agent [Yang et al. \(2024b\)](#) and OpenDevin [Wang et al. \(2024\)](#) act as autonomous developers, capable of understanding bug reports, navigating codebases, writing code, and executing tests to resolve complex issues. Other works have focused on specialized tasks like automated program repair [Xia et al. \(2023\)](#); [Monperrus \(2018\)](#); [Zhao et al. \(2023\)](#); [Fan et al. \(2023\)](#); [Rondon et al. \(2025\)](#); [Zhang et al. \(2024b\)](#); [Liu et al. \(2024\)](#) and test-free fault localization [Ni et al. \(2023\)](#); [Qin et al. \(2024\)](#). This body of work provides compelling evidence that LLMs can perform complex, multi-step tasks within the software domain. However, their focus has overwhelmingly been on generative tasks—creating or modifying code. SRE work, in contrast, is primarily investigative and diagnostic. It requires a different cognitive workflow centered on hypothesis generation, evidence gathering, and logical deduction. OpenDerisk adapts the proven power of LLMs from generative SE to the specific diagnostic challenges inherent in SRE.

Multi-Agent Frameworks. As tasks grow in complexity, the "divide and conquer" strategy of multi-agent systems has proven superior to monolithic agent designs [Li et al. \(2024; 2023\)](#); [Rizk et al. \(2020\)](#); [Finin et al. \(1994\)](#); [Dong et al. \(2024\)](#). General-purpose frameworks like AutoGen [Wu et al. \(2023\)](#), MetaGPT [Hong et al. \(2024\)](#), TaskWeaver [Qiao et al. \(2024\)](#), SEEAgent [Bui et al. \(2025\)](#) and others orchestrate teams of LLM agents to collaboratively solve problems, from writing entire software projects to complex data analysis. These systems have pioneered sophisticated collaboration patterns, role assignments, and communication protocols. However, their collaboration models are typically optimized for software construction or general problem-solving, not for the structured, high-stakes investigation of a system outage. More domain-specific systems like OpenRCA [Xu et al. \(2025\)](#)

¹The open-source project is available at: <https://github.com/derisk-ai/OpenDerisk>

apply the multi-agent paradigm to SRE but often employ a generalist problem-solving model that struggles to coordinate the deep, specialized expertise required for industrial incidents. OpenDeRisk distinguishes itself by implementing a diagnostic-native collaboration model, where specialized agents (e.g., a "database expert," a "network analyst") work together in a manner that mirrors an expert SRE team during an incident investigation.

Context Engineering and Optimization. The performance of any LLM-based system is critically dependent on its ability to manage the finite context window [Pinto et al. \(2024\)](#); [Manus \(2025\)](#). A significant body of research addresses this challenge. Retrieval-Augmented Generation (RAG) [Lewis et al. \(2020\)](#) has become the standard for injecting external, up-to-date knowledge into the context. Reasoning techniques like Chain-of-Thought (CoT) [Wei et al. \(2022\)](#) and ReAct [Yao et al. \(2023\)](#) structure the context to elicit more robust, step-by-step reasoning. Furthermore, frameworks like Toolformer [Schick et al. \(2023\)](#) and Gorilla [Patil et al. \(2023\)](#) have established methods for teaching LLMs to use external tools via API calls represented in their context. While these are powerful, foundational techniques, they are often applied generically. SRE diagnostics require managing a long-running, evolving context filled with heterogeneous data (logs, metrics, traces, alerts, documentation) and a dynamic set of tools. A simple ReAct loop or RAG query is insufficient. OpenDeRisk addresses this with its Model Context Protocol (MCP), a specialized form of context engineering that provides a standardized, stateful structure for managing the complex interplay of observations, thoughts, actions, and tool outputs throughout a protracted diagnostic session.

In summary, OpenDeRisk synthesizes advances from these four domains to create a novel solution. It leverages the diagnostic power of LLMs, organizes them within a specialized multi-agent framework tailored for SRE workflows, and governs their operation through a sophisticated context engineering protocol, thereby overcoming the limitations of prior work.

3 OpenDeRisk System

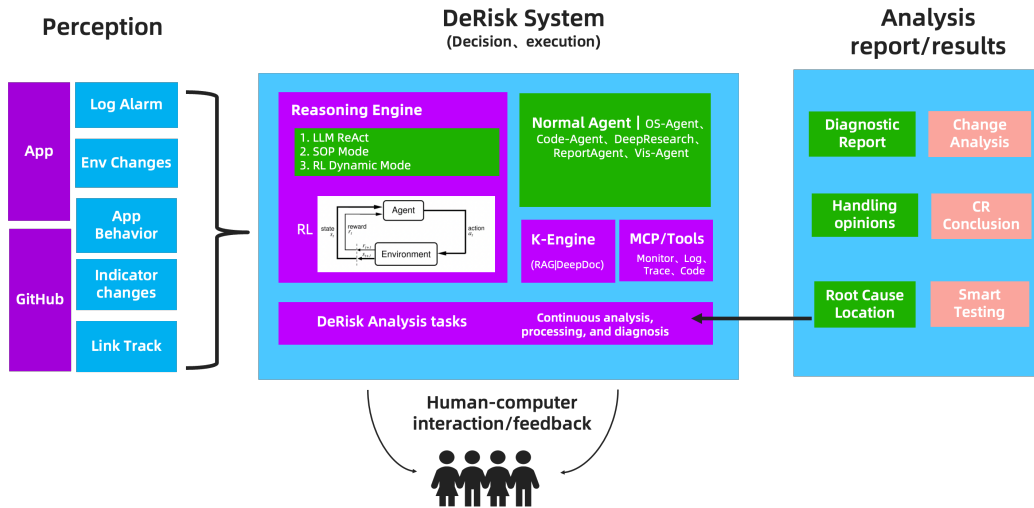


Figure 2: The end-to-end architecture of the OpenDeRisk system, illustrating the flow from Perception to Analysis, centered around the core Decision and Execution engine, and augmented by Human-in-the-Loop feedback.

This section details the architecture and core components of the OpenDeRisk framework, which is designed as an end-to-end diagnostic pipeline that mimics a human expert’s problem-solving process. We first present the design philosophy that underpins the entire system. Then, following the logical flow depicted in Figure 2, we describe the system’s three primary stages: a *Perception*

Layer to sense the environment, a core *DeRisk System* for decision-making and execution, and an *Analysis Reporting Layer* to deliver actionable insights.

3.1 Design Philosophy

The architecture of OpenDeRisk is a deliberate choice, resulting from an evaluation of three dominant technical paradigms. We dismissed the traditional *Workflow* paradigm due to its inherent rigidity. While we recognize the immense future potential of *Agentic Reinforcement Learning* (Agentic RL), its current technical immaturity and high resource requirements make it impractical for present-day deployment. Therefore, OpenDeRisk is strategically built upon the **Multi-Agent ReAct** paradigm. This approach leverages the advanced reasoning of LLMs to create a flexible, intelligent, and collaborative system best aligned with modern engineering principles for building evolvable AI systems.

This philosophy is realized through three primary optimization objectives:

- **Multi-Agent Collaboration Optimization:** To continuously discover and refine the most effective collaboration paradigms among agents.
- **Context Engineering Optimization:** To treat context generation as a formal optimization problem, maximizing the output quality of the LLM.
- **System-Level Reinforcement Learning:** To view the former two as local optimizations, with the ultimate goal of enabling end-to-end systemic evolution through online learning.

3.2 System Workflow and Components

3.2.1 Perception Layer: Sensing the Environment

The system's diagnostic workflow begins at the Perception Layer, which is responsible for ingesting a diverse array of signals that indicate a potential or ongoing incident. These signals are sourced from various operational and development platforms, including: Log Alarms, anomalous App Behavior, Environment Changes, and code changes from GitHub. These perceptual inputs serve as the initial trigger for the core DeRisk System.

3.2.2 The DeRisk System: Core Decision and Execution

The DeRisk System is the central nervous system of the framework, where raw perceptual data is transformed into intelligent decisions and actions. This is where the core DeRisk Analysis tasks—continuous analysis, processing, and diagnosis—take place. It is composed of several tightly integrated components:

Multi-Agent System and Collaboration Paradigms OpenDeRisk employs a multi-agent framework to enable a sophisticated division of labor. As detailed in Figure 3, the framework orchestrates a team of specialized agents (e.g., OS-Agent, Code-Agent) and dynamically adapts its collaboration strategy (e.g., 'TeamMode', 'GroupMode') based on the complexity of the application scenario.

Reasoning Engine At the heart of each agent lies a pluggable Reasoning Engine, the cognitive core responsible for planning and decision-making. It supports multiple modes of operation, including dynamic LLM ReAct Mode for exploratory analysis, deterministic SOP Mode (Standard Operating Procedure Mode) for routine tasks, and an RL Dynamic Mode for self-optimization.

Knowledge Engine (K-Engine) To empower agents with deep domain expertise, the framework integrates a powerful Knowledge Engine (K-Engine). This component implements an advanced Retrieval-Augmented Generation (RAG) system by transforming raw enterprise data into an actionable knowledge base through the five-stage pipeline shown in Figure 4.

The pipeline operates as follows:

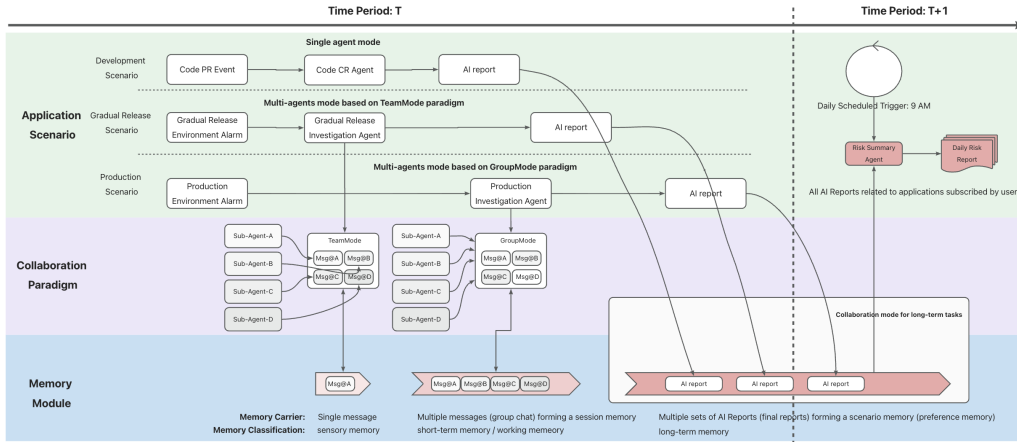


Figure 3: Multi-Agent Collaboration workflow in OpenDeRisk. The framework dynamically selects a collaboration paradigm (Single-Agent, TeamMode, or GroupMode) based on the application scenario, supported by a multi-layered memory module.

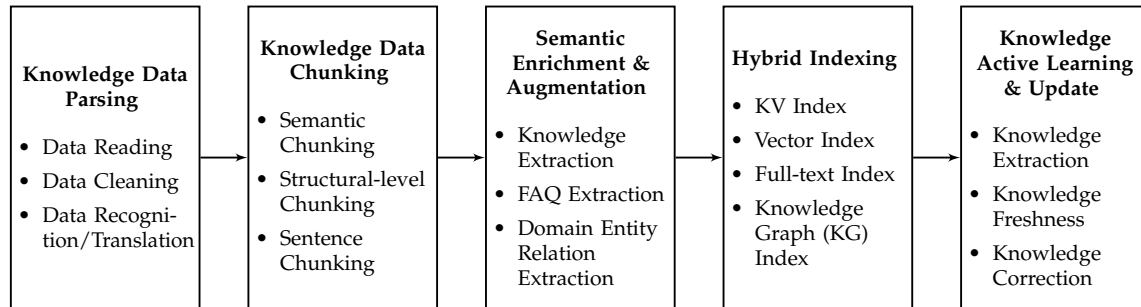


Figure 4: The five-stage knowledge processing pipeline of the K-Engine, which transforms raw data into a structured, indexed, and actively maintained knowledge base.

1. **Knowledge Data Parsing:** The process begins by ingesting data from various sources. This stage involves reading the raw data, performing data cleaning to remove noise and inconsistencies, and using data recognition/translation to standardize diverse formats into a unified representation.
2. **Knowledge Data Chunking:** To prepare the data for LLM consumption, the cleaned text is broken down into smaller, semantically coherent units. The engine employs multiple strategies, including *semantic chunking*, *structural-level chunking* (e.g., respecting code blocks or table boundaries), and basic *sentence chunking*.
3. **Semantic Enrichment & Augmentation:** Each chunk is then enriched to improve retrieval relevance. This involves extracting key knowledge, generating potential FAQ pairs, and identifying and tagging domain-specific entities and their relationships.
4. **Hybrid Indexing:** Recognizing that no single index is optimal, the engine constructs a hybrid index to support multi-faceted queries. This includes a **KV Index** for fast lookups, a **Vector Index** for semantic search, a **Full-text Index** for precise keyword matching, and a **Knowledge Graph (KG) Index** to model complex, multi-hop relationships.
5. **Knowledge Active Learning & Update:** Finally, to ensure the knowledge base does not become stale, an active learning loop continuously maintains it. This involves ongoing knowledge extraction from new data, checks for knowledge freshness, and knowledge correction based on feedback or newly discovered information.

This comprehensive process ensures that the K-Engine provides agents with a deep, reliable, and up-to-date understanding of the operational environment.

Tools and Model Context Protocol (MCP) Agents interact with the live environment through a standardized set of **Tools** for tasks such as monitoring and log analysis. The **Model Context Protocol (MCP)** is a formal contract that defines how tools are described and used, ensuring the system is highly extensible.

3.2.3 Analysis and Reporting Layer

The final stage of the pipeline is the Analysis and Reporting Layer, which synthesizes the system's findings into human-readable outputs. This layer produces artifacts such as Diagnostic Reports, Root Cause Location, and Handling opinions.

3.2.4 Human-in-the-Loop Feedback

A cornerstone of the OpenDerisk design is the continuous Human-in-the-Loop (HITL) feedback mechanism. At any point, an SRE can interact with the system to provide guidance or corrections. This ensures safety and provides valuable data for the RL training loop, allowing the system to learn from expert intervention.

4 Implementation

We have implemented the OpenDerisk framework as a robust and scalable system using a standard Python-based ecosystem. The entire framework is open-sourced to encourage further research and community contribution. Our implementation primarily utilized the Deepseek, Qwen, Claude and GPT models. This section details the key engineering decisions and components of our implementation, focusing on our context engineering strategies, agent orchestration, and visualization front-end.

4.1 Context Engineering Engine

The finite context window of LLMs is a primary bottleneck for complex, multi-step agent reasoning [Pinto et al. \(2024\)](#); [Manus \(2025\)](#); [Mei et al. \(2025\)](#). Our implementation addresses this through a sophisticated context engineering engine designed to maintain a high-signal, low-noise context. This engine is built on three core mechanisms:

- **Summary-Based Memory Compression:** As an agent's working memory (i.e., the history of its actions and observations) grows, the engine employs a compression strategy. Older turns in the conversation history are distilled into concise summaries, preserving key insights while freeing up token space. This allows the agent to maintain a long-term understanding of the task without exceeding the context limit.
- **Configurable Context Policies:** Recognizing that not all context is equally important, the engine supports configurable policies for context construction. An administrator can define rules that dictate how different parts of the context are handled. For example, a policy might specify that a user's original query and the agent's final reasoning steps are preserved in full, while intermediate raw tool outputs are heavily truncated or summarized. This provides granular control over the context payload.
- **Structured Report Distillation:** To prevent information distortion as reports from sub-agents are passed up to a supervisor (a common issue in multi-agent systems), our framework implements a structured distillation process. Instead of passing a full, verbose output, a sub-agent generates a structured summary object. This object contains key findings, confidence scores, and pointers to critical evidence, ensuring the supervisor receives a high-signal, low-noise summary and mitigating the risk of "lost-in-the-middle" errors.

Agent Type	Primary Responsibility	Key Capabilities
SRE-Agent	Site Reliability Engineering and orchestration	Incident coordination, system monitoring, agent orchestration
Code-Agent	Dynamic code generation and analysis	Runtime code generation, static analysis, code-based diagnostics
Data-Agent	Data processing and analysis	Log analysis, trace processing, metrics evaluation
Vis-Agent	Visualization and evidence presentation	Evidence chain visualization, diagnostic flow rendering
ReportAgent	Report generation and documentation	Diagnostic report creation, findings summarization

Table 1: Roles and capabilities of default digital employees (agents) in OpenDeRisk

Together, these mechanisms transform the context from a static transcript into a dynamically managed workspace, enabling sustained reasoning and the ability to perform cognitive refactoring by revising strategy based on a condensed, relevant history.

4.2 Agent Orchestration

The framework’s logic layer is implemented as a multi-agent architecture orchestrated by a central Orchestrator module. This module manages agent lifecycles, message passing, and task delegation. Each specialized agent (e.g., SRE-Agent, Code-Agent, Data-Agent) is implemented as a class inheriting from a base Agent class, which standardizes its core components: a **Profile** defining its role and expertise, a set of **Tools** representing its capabilities, a **Planning** engine (LLM-powered) for reasoning, a **Memory** module for maintaining state and context. Inter-agent communication is handled via an asynchronous message bus, allowing for parallel and scalable task execution. The entire system is designed for extensibility, allowing new agents to be seamlessly integrated to tackle emerging SRE challenges.

Case Study: How to Build a New Agent

We will use a real-world industrial case study "Emergency Time-Series Trend Analysis" to illustrate the core concepts, from initial design to a sophisticated, multi-agent validation system.

The Business Problem: During incident response, SREs heavily rely on monitoring time-series curves to determine if a system’s state is improving, worsening, or stable. The goal was to create a OpenDeRisk’s agent that can analyze a given monitoring curve and autonomously determine.

The initial implementation focuses on creating a two-agent team: an `Analysis_Agent` to perform the core task and a `Summarizer_Agent` to format the final report.

Step 1: Building the `Summarizer_Agent` We start with the simplest component. This agent’s sole purpose is to take the findings from other agents and generate a well-formatted report.

- **Agent Type:** ‘Task-based Agent’.
- **Reasoning Engine:** We select the `Summarizer Reasoning Engine` in the OpenDeRisk tools lib, which is optimized for digesting context and generating reports.
- **Tools & Knowledge:** None required.
- **System Prompt:** A prompt is configured to define the desired output format, instructing the agent to create a clear, concise report based on the upstream analysis.

Step 2: Building the `Analysis_Agent` (The “Master”) This agent is responsible for the main logic.

- **Agent Type:** ‘Task-based Agent’ with the `Default Reasoning Engine`.
- **Tools:** We bind the `mcp-antmonitor` tool, giving the agent the ability to fetch time-series data from the monitoring system.
- **Sub-Agents:** We link the `Summarizer_Agent` created in the previous step.

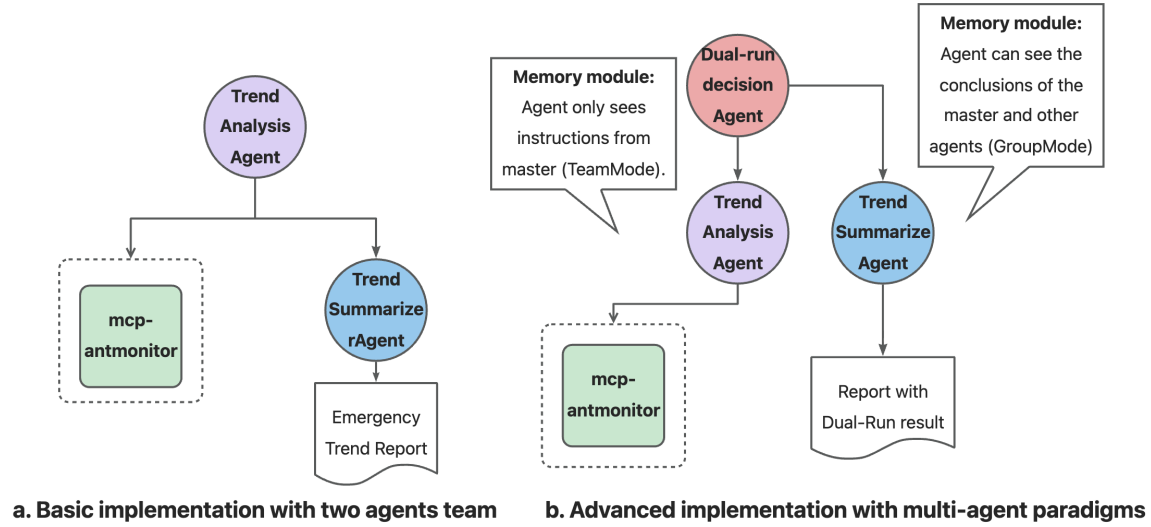


Figure 5: The designs of Emergency Time-Series Trend Agent. Left is basic version with a two-agents team, right is an advanced design with multi-agents paradigm.

- **System Prompt (Workflow Definition):** The prompt instructs the agent on its workflow: (1) Receive an alert and time window; (2) Use the mcp-antmonitor tool to retrieve data; (3) Analyze the trend; (4) Pass the data and conclusion to the Summarizer_Agent.

Step 3: Testing and Deployment (V1) We test the system with a real-world task query:

“Alert on anonymouseapp for ‘error rate’. Start time: 2025-08-19 15:21:00. Analyze the monitoring curve trend as of 15:26:00. Is it worsening or recovering?”

The agent successfully fetches the data, correctly identifies the trend, and generates a report with a plotted curve, fulfilling the initial goal, as shown in Figure 6. That means the agent works.

Step 4: Building an Advanced Agent To validate our agent, we designed an advanced dual-run agent to perform a non-biased comparison against a legacy system’s conclusions proposed by basic version. The core challenge is to prevent our new Analysis_Agent from seeing the legacy answer, which would bias its analysis. This is achieved by leveraging OpenDerisk’s advanced features, including a three-agent architecture orchestrated by a DualRun_Orchestrator. This orchestrator provides a sanitized task to the Analysis_Agent, which is placed in an isolated TeamMode memory to perform a blind analysis. Finally, a Critic_Summarizer_Agent operating in GroupMode gains access to the full context—including both the legacy and new conclusions—to generate a fair comparison report, thus enabling a robust, industrial-grade validation workflow.

4.3 Visualization and Human-in-the-Loop Interaction

To provide critical transparency into the framework’s complex internal processes, OpenDerisk includes a web-based user interface that serves as a real-time visualization front-end (Figure 7). This interface is not merely a static dashboard; it is powered by a custom *Visualization Protocol* designed to stream and render the entire problem-solving journey dynamically.

Leveraging this protocol, the UI makes the agent’s “thinking” process tangible by rendering a comprehensive, step-by-step view of:

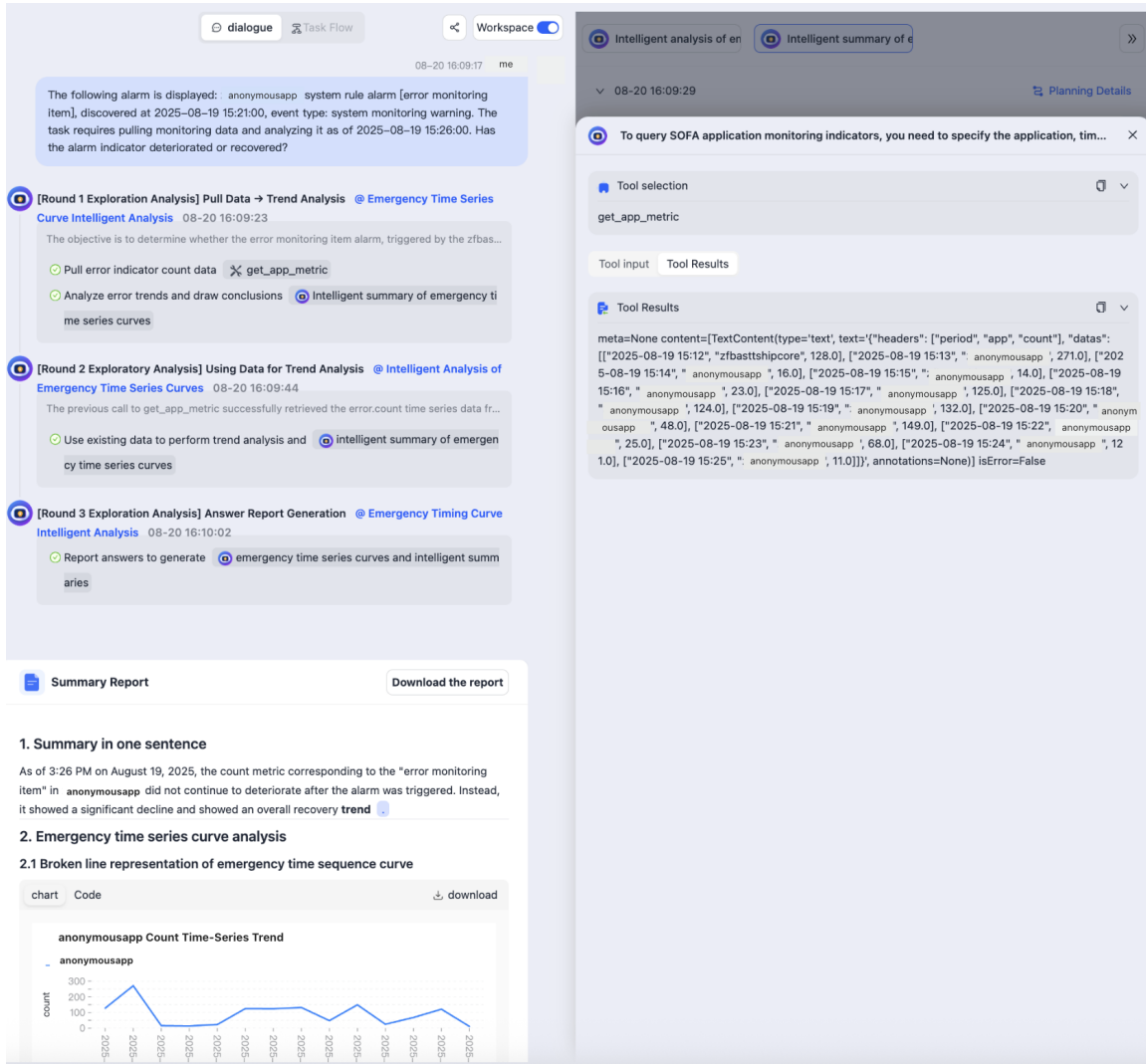


Figure 6: Test Agent—left panel: query and workflow; right panel: sequential execution results. Displayed is the return value from mcp-antmonitor's get_app_metric.

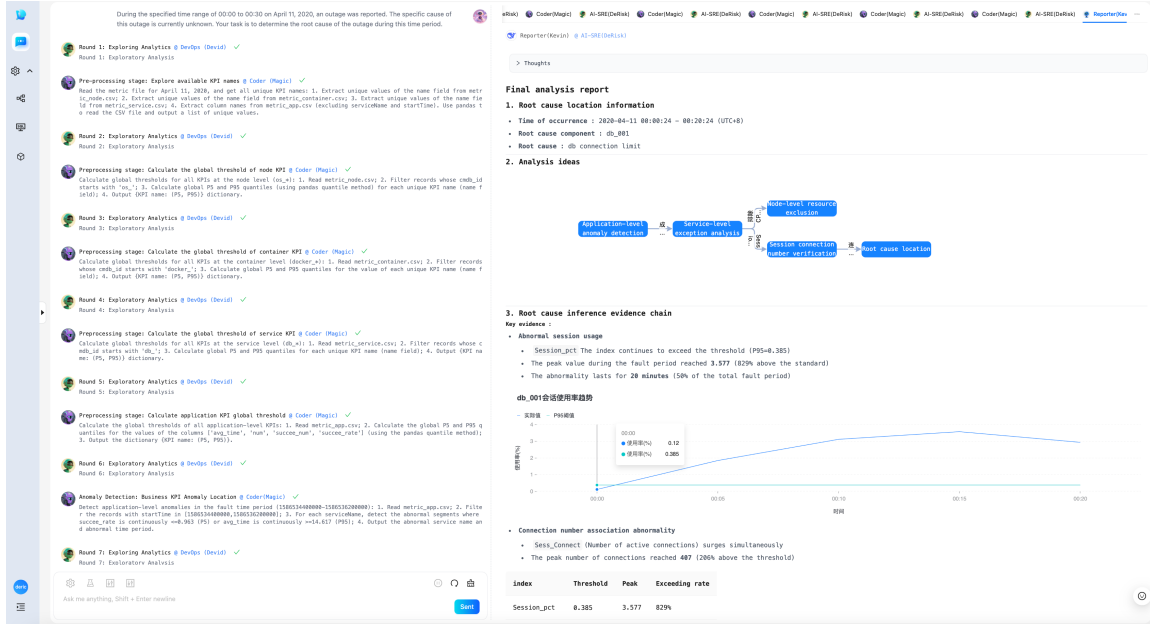


Figure 7: The user interface of OpenDerisk, visualizing the agent's multi-step reasoning and collaboration process.

- **The Reasoning Flow:** The logical path the supervisor agent takes to decompose and solve the problem.
- **The Evidence Chain:** The specific data (alerts, logs, traces) collected and analyzed by each specialized agent.
- **Multi-Agent Dynamics:** The explicit collaboration and role-switching between different agents as the investigation progresses.

This dynamic visualization is essential for building user trust and enabling human-in-the-loop oversight. It allows SREs to not only see the final conclusion but to understand how the system arrived at it, making the AI's diagnostic process fully auditable and intelligible.

4.4 Deployment at Ant Group

To validate its practicality and scalability in a real-world setting, the OpenDerisk framework has been deployed in production at Ant Group. The deployment validates a core design tenet of the framework: to serve as a general-purpose AI-SRE platform that empowers developers to autonomously create new agents for diverse SRE tasks, extending far beyond the initial focus on Root Cause Analysis.

The results from this deployment underscore the framework's real-world impact and rapid adoption. Over a three-month production period:

- **Extensibility and Adoption:** The framework has been applied to **13 new application scenarios**. Internal developers have successfully created over **50 new specialized agents** to address their unique operational needs, confirming the framework's ease of extension.
- **Scale and Usage:** The platform has achieved significant scale, serving over **3,000 daily active users** and executing more than **60,000 diagnostic runs per day**.

This large-scale, sustained usage in a mission-critical environment demonstrates that OpenDerisk is not merely a research prototype but a robust, scalable, and valuable industrial-grade system.

Building on the success of this internal system, the core technology has been productized into a commercial-grade platform.

5 Evaluation

5.1 Methodology

Our evaluation is structured around three central research questions (RQs) designed to assess the framework’s key attributes, from its overall efficiency to the contribution of its core architectural components. To answer these questions, we compare three primary evolutionary stages of our agent framework, each designed to overcome the limitations of its predecessor.

The Research Questions are as follows:

- **RQ1 (Efficiency and Effectiveness):** *How does our multi-agent architecture compare to monolithic agents in terms of accuracy and execution time on complex industrial tasks?*
- **RQ2 (Adaptability):** *How effectively does the framework integrate new, domain-specific knowledge and adapt to different foundational LLMs?*
- **RQ3 (Ablation and Contribution):** *What is the specific contribution of each architectural enhancement to the agent’s performance?*

To address these questions, we evaluate the following system configurations:

V1: Basic ReAct Agent. This baseline represents a single agent operating in a simple Think -> Act -> Observe loop. While capable of basic tool use, this architecture lacks robust end-to-end control and is prone to deviating from the main task when faced with complexity.

V2: Phased-Control ReAct Agent. This is a more structured, yet still monolithic, single-agent architecture. It introduces a predefined sequence of problem-solving stages (e.g., *Root Cause Analysis*, *Internal Cause Analysis*) and uses a Dynamic Prompt Control mechanism to constrain the agent’s behavior at each phase. However, this rigid structure can be brittle to long-chain hallucinations and is sensitive to context window limitations.

V3: Multi-Specialist Agent Framework (Ours). This is our proposed “divide and conquer” paradigm. A central **Supervisor** agent decomposes the primary task. It then delegates sub-tasks to the most appropriate component: it uses `ToolCall` to trigger a **Workflow Engine** for deterministic tasks and `Handoff` to dispatch complex problems to specialized **Sub-Agents**. This design dramatically reduces the reasoning burden on any single agent and improves accuracy by leveraging targeted expertise. By comparing V3 against V1 and V2, we can quantify the performance gains and isolate the contribution of each architectural evolution.

5.2 Case Study 1: Root Cause Analysis

5.2.1 Experiment Design

To empirically evaluate our architectural progression, we designed a comparative experiment measuring the trade-offs between task success and execution time. We compare three distinct agent architectures, each powered by a diverse set of Large Language Models (LLMs) to validate model-agnosticism.

To demonstrate the generality of our architectural improvements, each configuration was tested with a diverse set of foundational LLMs, including Qwen-QWQ-32B [Yang et al. \(2024a\)](#); [Team \(2025\)](#),

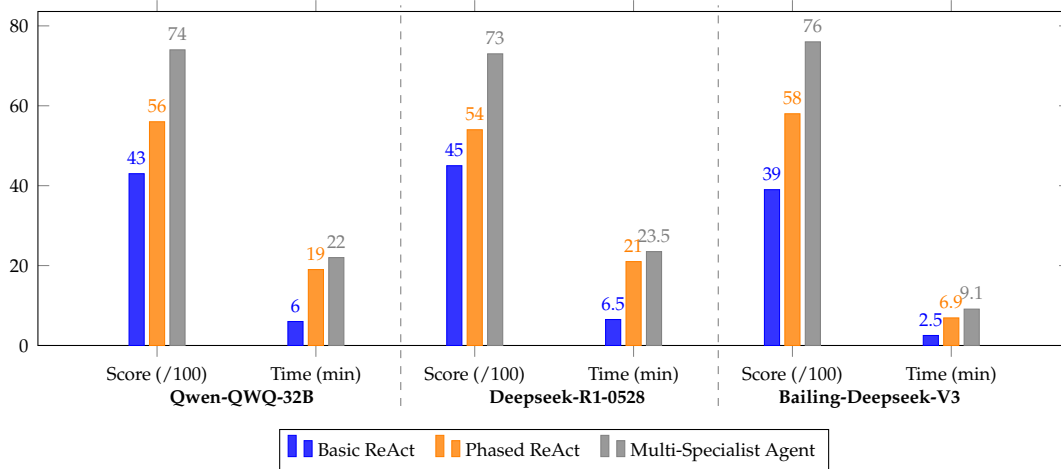


Figure 8: Performance Comparison of Agent Frameworks Across Different Models.

Deepseek-R1-0528 [DeepSeek-AI et al. \(2025\)](#), and Bailing-Deepseek-V3 [Di et al. \(2024\)](#); [Codefuse et al. \(2025\)](#)².

We evaluated all methods on a hybrid benchmark combining the public **OpenRCA** dataset (335 scenarios) and a proprietary **AntRCA** dataset. The AntRCA dataset was curated from real-world cases at Ant Group and its composition was adjusted to mirror the true distribution of daily SRE tasks, ensuring high industrial relevance. Performance was quantified using a human-graded 100-point scoring rubric.

5.2.2 Results and Analysis

Experimental results, shown in Figure 8, confirm a consistent performance hierarchy (V3 > V2 > V1) across all tested LLMs, demonstrating the model-agnostic architectural superiority of our Multi-Specialist Agent framework.

Improvement in Task Accuracy Across all three base models, Qwen-QWQ-32B, Deepseek-R1-0528, and Bailing-Deepseek-V3, the V3 framework consistently achieved the highest scores. For instance, with the Bailing-Deepseek-V3 model, the Multi-Specialist Agent scored **76**, a substantial improvement over the Phased ReAct’s score of **58** and the Basic ReAct’s score of **39**. This significant leap in performance is attributed to the *divide and conquer* strategy of the V3 architecture. The Supervisor agent effectively decomposes the complex problem into smaller, manageable sub-tasks, which are then handled by specialized agents. This stands in stark contrast to the V1 agent’s monolithic and often unfocused reasoning, and the V2 agent’s rigid, sequential process, which is vulnerable to cascading errors.

The Accuracy-Efficiency Trade-off This improvement in accuracy, however, comes at the cost of increased execution time. The results show that the V3 framework is consistently the slowest, while the V1 framework is the fastest. For example, using Qwen-QWQ-32B, the execution time increased from a mere **6 minutes** for the V1 agent to **22 minutes** for the V3 framework. This latency is an expected consequence of the V3 architecture’s increased complexity, which involves additional reasoning steps for task decomposition by the Supervisor and inter-agent communication overhead. The V2 architecture represents a middle ground in both accuracy and time.

²Bailing-Deepseek-V3 is an in-house model developed by Ant Group’s CodeFuse team by fine-tuning Deepseek-V3.

Conclusion for RQ1 In conclusion, the Multi-Specialist Agent framework successfully navigates the accuracy-efficiency frontier. It demonstrates that by investing more computational time in a structured, multi-agent reasoning process, it can achieve a dramatic and previously unattainable level of accuracy on complex industrial tasks. The increased latency is a direct and worthwhile trade-off for this substantial gain in problem-solving capability.

5.3 Case Study 2: Trace Analysis

5.3.1 Experimental Design

This experiment is designed to directly address **RQ2: How effectively does the framework integrate new, domain-specific knowledge and adapt to different foundational LLMs?** To answer this, we structured our evaluation to systematically test both aspects of the question: knowledge integration and model adaptability.

- **Validating Domain-Specific Knowledge Integration:** We first addressed the “Knowledge Gap”—the absence of Ant Group’s proprietary operational knowledge in public LLMs. Using the OpenDerisk framework, we rapidly developed a new trace-based RCA agent by injecting this expert knowledge. To measure the effectiveness of this integration, the agent was deployed into production at Ant Group for a **one-month trial**. Its performance was validated by **1,743 developers** across over **6,000 real-world cases**, providing a robust, in-situ test of its practical capability.
- **Assessing Adaptability to Foundational LLMs:** Next, we tested the framework’s model-agnosticism. We leveraged the OpenDerisk architecture to systematically integrate and evaluate a diverse set of foundational LLMs (as detailed in Table 2). This comparative analysis on a curated dataset allowed us to quantify how well the framework adapts to different models and identify which are best suited for the task.

Evaluation Metrics To quantify the outcomes for both objectives, performance was measured across three axes:

- **Aggregate Success Rate:** The percentage of cases where the agent correctly identified the root cause, directly measuring the effectiveness of knowledge integration (Target: >80%).
- **User-Rated Analysis Quality:** A 1-to-5 star rating from developers, reflecting the perceived utility and quality of the agent’s analysis.
- **System Execution Time:** The end-to-end latency, serving as a critical metric for comparing the operational efficiency of different LLMs.

5.3.2 Results and Analysis

The results in Table 2 provide the answer to RQ2 by demonstrating the OpenDerisk framework’s dual capabilities: effective integration of domain-specific knowledge and seamless adaptability across diverse foundational LLMs.

Effective Knowledge Integration The primary success metric—an **aggregate success rate exceeding 80%** across 6,000+ cases in the one-month production trial—serves as direct validation for the first part of RQ2. This result proves that the framework successfully bridged the “Knowledge Gap.” By providing the tooling to rapidly inject proprietary knowledge from Ant Group, OpenDerisk transformed general-purpose LLMs into high-performing, specialized agents. The high success rate and top-tier *Analysis Quality* scores for models like GPT-4o and Deepseek-v3 show that this injected knowledge translates directly into practical, production-grade problem-solving capability.

Model Adaptability and Strategic Tunability The breadth of models evaluated in Table 2 provides a clear answer to the second part of RQ2. The framework’s model-agnostic, “plug-and-play”

Table 2: Performance Comparison of LLMs Integrated via the Derisk Framework for Trace-based RCA

Model	Model Type	Correct Rate (>80%)	Analysis Quality	Time
GPT-oss-120B	Reasoning	✓	★★★★★	Fast
GPT-oss-20B	Reasoning	✓	★★★★★	Fast
GPT-4o	Non-Reasoning	✓	★★★★★	Very Fast
Deepseek-v3	Non-Reasoning	✓	★★★★★	Fast
Deepseek-r1	Reasoning	✗	★★	Very Slow
Kimi-K2	Non-Reasoning	✓*	★★★★	Fast *
Kimi-K1.5	Non-Reasoning	✓	★★★★	Slow
QWQ-32B	Reasoning	✓	★★★	Very Slow
Qwen3-30B	Non-Reasoning	✗	★	Fast
Qwen3-235B	Non-Reasoning	✓	★★★★★	Fast
Qwen3-Coder	Non-Reasoning	✗	★	Fast
Qwen-2.5	Non-Reasoning	✗	★	Fast
Gemini-pro-2.5	Reasoning	✓	★★★★	Slow
Gemini-flash-2.5	Non-Reasoning	✓	★★★★	Slow
Claude-Sonnet-3.5	Non-Reasoning	✓	★★★★★	Fast

* denotes Kimi-K2, which exhibited inconsistent performance across multiple runs. Its correctness rate fluctuated above and below 80%, with execution times occasionally exceeding 30 seconds.

architecture was instrumental in this large-scale comparative analysis. This adaptability is not merely a technical feature but a crucial enabler of strategic tunability. It allows for empirical, data-driven decisions that balance performance, cost, and latency.

For instance, the benchmark data clearly illuminated critical trade-offs: while Deepseek-R1 failed to meet performance requirements due to its slow speed, the faster Deepseek-v3 emerged as a superior production candidate without sacrificing quality. This data-driven insight, enabled by the framework, allowed us to confidently select Deepseek-v3 for the production trial—a decision ultimately validated by the high user-rated quality scores from our 1743 developers. Furthermore, the framework facilitated the identification of model-specific weaknesses, such as the difficulty some Qwen-series models had with JSON parsing, effectively de-risking the model selection process.

Conclusion for RQ2 In conclusion, this comprehensive evaluation demonstrates that OpenDerisk excels in its core design goals. It serves as a robust meta-platform that provides both the agility to build domain-specific agents and the model-agnostic flexibility to empirically evaluate, select, and deploy them in a demanding industrial environment.

5.4 Ablation Study

To answer RQ3 and quantify the contribution of our core architectural mechanisms, we conducted a qualitative ablation study. The evaluation, detailed in Table 3, is grounded in a set of representative business-logic-related cases selected from our AntrCA (used in Case Studies 1 and 2) and live production incidents. The performance across each dimension was calibrated by SRE experts to ensure a consistent and accurate assessment.

V1 (Basic ReAct): A Brittle Baseline The V1 agent, representing a standard ReAct implementation, served as our baseline. As the table shows, its capabilities are severely limited. In the “Currency is null” case, its analysis is “Shallow (code only),” and for the “Riskcloud policy” case, it completely fails at “Code Localization.” This confirms that a simple, unstructured agent loop is insufficient for complex industrial tasks; it lacks the depth to understand business logic and the robustness to navigate multi-step causal chains.

Table 3: Comprehensive Qualitative Comparison of Agent Architectures (V1-V3)

Case Description	Dimension	V1: Basic ReAct	V2: Phased-Control ReAct	V3: Multi-Specialist
Currency is null	Localization Accuracy	★★★★ Basic	★★★★ Precise	★★★★ Complete
	RCA Depth	Shallow (code only)	Medium (business logic)	Deep (configuration)
	Runtime Data Completeness	Missing key params	Partially inferred data	Complete actual data
The riskcloud policy engine returned anti-fraud flag with REJECT	Impact Assessment	Simple exclusion	Not addressed	Identifies upstream factors
	Operability	Low	Medium	High
Error code mapping wrong rules	Raw Evidence Collection	★★★★ Complete	★★★★ Mostly Complete	★★★★ Complete
	Causal Chain Analysis	★★ Lacking	★★★★ Clear	★★★★ In-depth
	Code Localization	✗ Failure	★★★★ Partially successful	★★★★ Precise
Security controller passed a REJECT decision	Technical Depth	★★ Shallow	★★★★ Medium	★★★★ In-depth
	Config. Query Mechanism	✗ Lacking	✗ Lacking	✓ Detailed query flow
	Fallback Logic	✗ Not addressed	✗ Not addressed	✓ Explicit error handling
Page is oversize	Config. Storage Location	✗ Vague	✗ Vague	✓ Pinpoints DATA_ITEM_NAME
	Causal Chain Depth	● Shallow	● Medium	✓ Deep analysis
Invalid member status	Architecture Understanding	✗ Assumes external deps	△ Single-module analysis	✓ Module-chain arch
	Causal Chain Completeness	✗ Breaks at external services	△ Lacks normalization step	✓ Complete closed-loop
	Operability	✗ Requires external team	△ Partially operable	✓ Internally diagnosable
Contract version issue	Default Threshold ID	✓ 100	✗ 200	✓ 100
	Core Code Localization	✗ Lacking	✓ Complete	✓ Complete
	Error Chain Completeness	✗ Incomplete	✓ Complete	✓ Complete
SRE Practicality	Log Evidence	✗ Lacking	✓ Complete	✓ Complete
	Issue Focus	✗ Off-topic	✓ Highly Focused	✓ Highly Focused
Key Findings	Report Format	Strict format, categorized by subject	Structured analysis, causal chain narrative	JSON output, organized by field
	Technical Depth	★★ Shallow reference	★★★★ Complete tech. chain	★★★★ Medium-deep
	Evidence Completeness	★★★★ Basic evidence complete	★★★★ Code + logs covered	★★★★ Rich but redundant
Core Issue Focus	Operability	★★ Needs secondary analysis	★★★★ Points to solution path	★★★★ Info overload, needs focus
	Key Findings	- Basic code localization - Monitoring data normal	3-step data transform - Specific code line (40) - Complete anomaly propagation	- Cross-stack dependency - Physical location info - Component version info
Recovery Support	SRE Practicality	★★ Insufficient info	★★★★ Best practice standard	★★★★ Information overload
	Core Issue Focus	★★ Focused but shallow	★★★★ Precisely checks FREEZE state	★★★★ Effectively filters irrelevant info
	Recovery Support	★★ Needs more analysis	★★★★ Directly supports solution	★★★★ Partially useful info
Localization Accuracy	Localization Accuracy	★★★★ Basic	★★★★ Precise	★★★★ Complete
	RCA Depth	Shallow (code only)	Medium (business logic)	Deep (configuration)
	Runtime Data Completeness	Missing key params	Partially inferred data	Complete actual data
Impact Assessment	Impact Assessment	Simple exclusion	Not addressed	Identifies upstream factors
	Operability	Low	Medium	High

V2 (Phased ReAct): The Value of Structured Reasoning and Experience The introduction of a predefined, phased workflow in V2 yields significant improvements. This enhancement is driven by two key mechanisms: first, V2 emulates a developer’s code walkthrough experience by generating an agent that performs taint analysis to explore potential data flow issues; second, its reasoning is augmented with domain-specific prompts optimized for business logic. These additions allow V2 to deliver a “Structured analysis” and follow a “Complete tech. chain” in the “Invalid member status” case, a capability far beyond V1’s shallow analysis. However, V2’s monolithic nature remains a bottleneck. It struggles with cross-domain analysis, as shown in the “Currency is null” case where it fails to perform an “Impact Assessment,” and in the “Security controller” case, it is confined to “Single-module analysis.” This highlights the limitations of a single agent trying to be a generalist.

V3 (Multi-Specialist): The Critical Contribution of Collaborative Specialization The transition to the V3 Multi-Specialist framework represents the most significant leap, directly addressing the limitations of the V2 agent. The “divide and conquer” paradigm proves to be the key mechanism for achieving production-grade analysis:

- **Depth and Breadth:** In the “Currency is null” case, V3 achieves “Deep (configuration)” analysis and “Identifies upstream factors.” This is possible because the Supervisor agent can delegate tasks to specialized agents (e.g., a Data-Agent for runtime data and a Config-Agent for configuration parameters), enabling a holistic, cross-stack view that a single agent cannot manage.
- **Architectural Awareness:** For the “Security controller” case, V3 demonstrates a true understanding of the “Module-chain arch.,” creating a “Complete closed-loop” analysis. This capability stems directly from the multi-agent collaboration, which can trace dependencies across service boundaries.
- **Precision and Focus:** In the “Error code mapping” case, V3 is able to execute a “Detailed query flow” and “Pinpoints DATA_ITEM _NAME,” showcasing the power of specialized tool use within the framework. Similarly, in the “Page is oversize” case, V3 remains “Highly Focused” where V1 was “Off-topic,” demonstrating the Supervisor’s role in maintaining task coherence.

Conclusion for RQ3 Our analysis confirms that while a structured single-agent (V2) provides a solid foundation, multi-specialist collaboration (V3) is the critical architectural leap that delivers the robustness and depth essential for enterprise-grade diagnostics.

6 Discussion and Limitations

Despite its promising results, OpenDerisk has several key limitations. First, it operates as an assistive “co-pilot” requiring human oversight for final remediation, rather than a fully autonomous system. Second, its diagnostic accuracy is fundamentally dependent on the quality and maintenance of the underlying knowledge base. Third, there is an inherent trade-off between analytical depth and performance, as the most accurate multi-agent approach incurs higher latency and computational cost. Finally, its generalizability beyond the specific and mature observability ecosystem of Ant Group has not yet been validated in organizations with different technology stacks or data quality.

7 Conclusion and Future Work

In this paper, we introduced OpenDerisk, a general-purpose AI-SRE framework designed not only for complex Root Cause Analysis (RCA) but also to empower developers to autonomously create new agents for a wide range of SRE tasks. Our primary contribution is the demonstration that a multi-specialist collaboration paradigm (V3) significantly outperforms monolithic agent architectures in diagnostic depth, accuracy, and robustness.

By integrating a sophisticated Knowledge Engine, adaptive collaboration patterns, and a pluggable Reasoning Engine, OpenDerisk provides a practical and powerful “co-pilot” for SRE teams. This

practicality and scalability are substantiated by its successful production deployment at Ant Group. Over a three-month period, the framework has been adopted for **13 new application scenarios**, with developers creating over **50 new specialized agents**. Today, it serves more than **3,000 daily users** and executes over **60,000 runs per day**. This real-world data, combined with our experimental results, validates our design philosophy and marks a significant step towards building truly intelligent, adaptable, and scalable diagnostic systems for enterprise environments.

Our roadmap focuses on evolving OpenDeRisk from a "co-pilot" to an autonomous "pilot" (V4) capable of executing safe, closed-loop remediation. This will be driven by a comprehensive Reinforcement Learning paradigm to optimize both individual agent tool-use and system-level collaboration strategies. We also plan to develop a self-healing Knowledge Engine, where agents contribute to their own knowledge base, and an adaptive reasoning controller to dynamically balance diagnostic depth against response latency.

References

- Andrew Arnold, Yan Liu, and Naoki Abe. Temporal causal modeling with graphical granger methods. In *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*, pp. 66–75, 2007.
- Joop Aué, Maurício Aniche, Maikel Lobbezoo, and Arie van Deursen. An exploratory study on faults in web api integration in a large-scale payment company. In *Proceedings of ICSE-SEIP '18: 40th International Conference on Software Engineering: Software Engineering in Practice Track*, 2018. doi: 10.1145/3183519.3183537.
- Len Bass, Ingo Weber, and Liming Zhu. *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional, 1st edition, 2015. ISBN 0134049845.
- LEN BASS, QINGHUA LU, INGO WEBER, and LIMING ZHU. *Engineering AI Systems: Architecture and DevOps Essentials*. Addison-Wesley, 2025.
- Tingzhu Bi, Zhang Yang, Yicheng Pan, Yu Zhang, Meng Ma, Xinrui Jiang, Linlin Han, Feng Wang, Xian Liu, and Ping Wang. Faultinsight: Interpreting hyperscale data center host faults. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 141–152, 2024.
- Thanh-Long Bui, Hoa Khanh Dam, and Rashina Hoda. An llm-based multi-agent framework for agile effort estimation, 2025. URL <https://arxiv.org/abs/2509.14483>.
- Sarthak Chakraborty, Shaddy Garg, Shubham Agarwal, Ayush Chauhan, and Shiv Kumar Saini. Causil: Causal graph for instance level microservice data. In *Proceedings of the ACM Web Conference 2023*, pp. 2905–2915, 2023.
- Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. volume 41, pp. 1–58. ACM, 2009.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique P Ponde, Karpathy Andrej, et al. Evaluating large language models trained on code. In *arXiv preprint arXiv:2107.03374*, 2021.
- Codefuse, Ling Team, :, Wenting Cai, Yuchen Cao, Chaoyu Chen, Chen Chen, Siba Chen, Qing Cui, Peng Di, Junpeng Fang, Zi Gong, Ting Guo, Zhengyu He, Yang Huang, Cong Li, Jianguo Li, Zheng Li, Shijie Lian, BingChang Liu, Songshan Luo, Shuo Mao, Min Shen, Jian Wu, Jiaolong Yang, Wenjie Yang, Tong Ye, Hang Yu, Wei Zhang, Zhenduo Zhang, Hailin Zhao, Xunjin Zheng, and Jun Zhou. Every sample matters: Leveraging mixture-of-experts and high-quality data for efficient and accurate code llm, 2025. URL <https://arxiv.org/abs/2503.17793>.
- Jeanderson Cândido, Jan Haesen, Maurício Aniche, and Arie van Deursen. An exploratory study of log placement recommendation in an enterprise system. In *Proceedings of Mining Software Repositories Conference (MSR)*, 2021. doi: 10.1109/MSR52588.2021.00027.
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu,

- Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanxia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Peng Di, Jianguo Li, Hang Yu, Wei Jiang, Wenting Cai, Yang Cao, Chaoyu Chen, Dajun Chen, Hongwei Chen, Liang Chen, Gang Fan, Jie Gong, Zi Gong, Wen Hu, Tingting Guo, Zhichao Lei, Ting Li, Zheng Li, Ming Liang, Cong Liao, Bingchang Liu, Jiachen Liu, Zhiwei Liu, Shaojun Lu, Min Shen, Guangpei Wang, Huan Wang, Zhi Wang, Zhaogui Xu, Jiawei Yang, Qing Ye, Gehao Zhang, Yu Zhang, Zelin Zhao, Xunjin Zheng, Hailian Zhou, Lifu Zhu, and Xianying Zhu. Codefuse-13b: A pretrained multi-lingual code large language model. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice, ICSE-SEIP '24*, pp. 418–429. ACM, April 2024. doi: 10.1145/3639477.3639719. URL <http://dx.doi.org/10.1145/3639477.3639719>.
- Liming Dong, Qinghua Lu, and Liming Zhu. Agentops: Enabling observability of llm agents, 2024. URL <https://arxiv.org/abs/2411.05285>.
- Gang Fan, Xiaoheng Xie, Xunjin Zheng, Yinan Liang, and Peng Di. Static code analysis in the ai era: An in-depth exploration of the concept, function, and potential of intelligent code analysis agents. *arXiv preprint arXiv:2310.08837*, 2023.
- Tim Finin, Richard Fritzson, Donald P McKay, Robin McEntire, et al. Kqml-a language and protocol for knowledge and information exchange. In *13th Int. Distributed Artificial Intelligence Workshop*, pp. 93–103, 1994.
- Pinjia He, Jieming Zhu, Shilin He, Jian Li, and Michael R Lyu. An evaluation study on log parsing and its use in log mining. In *2016 46th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pp. 654–661. IEEE, 2016.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xianwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VtmBAGN7o>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Gustavo Nogueira, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pp. 9459–9474, 2020.
- Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society. *Advances in Neural Information Processing Systems*, 36:51991–52008, 2023.
- Mingjie Li, Zeyan Li, Kanglin Yin, Xiaohui Nie, Wenchi Zhang, Kaixin Sui, and Dan Pei. Causal inference-based root cause analysis for online service systems with intervention recognition. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 3230–3240, 2022a.

- Pengfei Li, Tianyi Wen, Yisong Chen, Hongyang Chen, Guoliang Zhang, Zhibin Zhang, and Wei Lin. CloudRCA: A root cause analysis framework for cloud native applications. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, pp. 217–232, 2022b.
- Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. A survey on llm-based multi-agent systems: workflow, infrastructure, and challenges. *Viciniagearth*, 1(1):9, 2024.
- Yizhou Liu, Pengfei Gao, Xinchun Wang, Jie Liu, Yexuan Shi, Zhao Zhang, and Chao Peng. Marscode agent: Ai-native automated bug fixing. *arXiv preprint arXiv:2409.00899*, 2024.
- Manus. Context engineering for ai agents: Lessons from building manus, 2025. URL <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>.
- Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhongzhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. A survey of context engineering for large language models, 2025. URL <https://arxiv.org/abs/2507.13334>.
- Martin Monperrus. Automatic software repair: a survey. *ACM Computing Surveys (CSUR)*, 51(1): 1–48, 2018.
- Ansong Ni, Chunqiu Liu, Zhendong Qian, Zichuan Liu, Yufan Lin, Kaixin Chen, Geguang Pu, and Chao Zhang. Leveraging large language models for test-free fault-localization. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023.
- Shishir G Patil, Tianjun Gadek, Joseph E Schneider, Swaroop Parmar, Donald He, Alon Weiss, et al. Gorilla: Large language model connected with massive apis. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Luan Pham, Huong Ha, and Hongyu Zhang. Baro: Robust root cause analysis for microservices via multivariate bayesian online change point detection. volume 1, pp. 2214–2237, 2024a.
- Luan Pham, Huong Ha, and Hongyu Zhang. Root cause analysis for microservice system based on causal inference: How far are we? In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pp. 706–715, 2024b.
- Luan Pham, Hongyu Zhang, Huong Ha, Flora Salim, and Xiuzhen Zhang. Rcaeval: A benchmark for root cause analysis of microservice systems with telemetry data. In *Companion Proceedings of the ACM on Web Conference 2025*, pp. 777–780, 2025.
- Gustavo Pinto, Cleidson de Souza, João Batista Neto, Alberto de Souza, Tarcísio Gotto, and Edward Monteiro. Lessons from building stackspot ai: A contextualized ai coding assistant, 2024. URL <https://arxiv.org/abs/2311.18450>.
- Bo Qiao, Liqun Li, Xu Zhang, Shilin He, Yu Kang, Chaoyun Zhang, Fangkai Yang, Hang Dong, Jue Zhang, Lu Wang, Minghua Ma, Pu Zhao, Si Qin, Xiaoting Qin, Chao Du, Yong Xu, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. Taskweaver: A code-first agent framework, 2024. URL <https://arxiv.org/abs/2311.17541>.
- Yihao Qin, Shangwen Wang, Yiling Lou, Jinhao Dong, Kaixin Wang, Xiaoling Li, and Xiaoguang Mao. Agentfl: Scaling llm-based fault localization to project-level context. *arXiv preprint arXiv:2403.16362*, 2024.
- Yara Rizk, Abhishek Bhandwalder, Scott Boag, Tathagata Chakraborti, Vatche Isahagian, Yasaman Khazaeni, Falk Pollock, and Merve Unuvar. A unified conversational assistant framework for business process automation. *arXiv preprint arXiv:2001.03543*, 2020.
- Pat Rondon, Renyao Wei, José Cambronero, Jürgen Cito, Aaron Sun, Siddhant Sanyam, Michele Tufano, and Satish Chandra. Evaluating agent-based program repair at google, 2025. URL <https://arxiv.org/abs/2501.07531>.

- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Tsvigun, Gwenaël Cances, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Peter Spirtes, Clark N Glymour, and Richard Scheines. *Causation, prediction, and search*. MIT press, 2000.
- Qwen Team. Qwq-32b: Embracing the power of reinforcement learning, March 2025. URL <https://qwenlm.github.io/blog/qwq-32b/>.
- Dongjie Wang, Zhengzhang Chen, Yanjie Fu, Yanchi Liu, and Haifeng Chen. Incremental causal graph learning for online root cause analysis. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*, pp. 2269–2278, 2023a.
- Peng-Cheng Wang, Yun-Fei Liu, Jia-Wei Zhou, Ling-Hao Cheng, Jun-Hao Zhou, Ze-Xuan Niu, Yin-Dong Li, Ying-Jie Chen, Chi-Shen Zhang, Si-Wei Sun, et al. OpenDevin: Evolving code generation with a test-driven agent, 2024.
- Yingzhe Wang, Zisheng Wang, Zipei Ma, Wenbin Zhu, Cong Wang, Jun Xu, Guangba Xu, Zeyu Wu, Jiwei Li, and Gang Zhao. MicroRCA: Root cause analysis of performance anomalies in microservice architecture. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 1660–1672. IEEE, 2023b.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pp. 24824–24837, 2022.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Li, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen llm applications via multi-agent conversation, 2023.
- Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. Automated program repair in the era of large pre-trained language models. In *ICSE’23*, 2023.
- Ruyue Xin, Peng Chen, and Zhiming Zhao. Causalrca: Causal inference based precise fine-grained root cause localization for microservice applications. *Journal of Systems and Software*, 203:111724, 2023. ISSN 0164-1212. doi: <https://doi.org/10.1016/j.jss.2023.111724>. URL <https://www.sciencedirect.com/science/article/pii/S016412122300119X>.
- Junjielong Xu, Qinan Zhang, Zhiqing Zhong, Shilin He, Chaoyun Zhang, Qingwei Lin, Dan Pei, Pinjia He, Dongmei Zhang, and Qi Zhang. OpenRCA: Can large language models locate the root cause of software failures? In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=M4qNizQYpd>.
- An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024a.
- John Yang, Carlos E. Jimenez, Xuan-Shi Li, Zhaojun Wang, Zixuan Cui, Zhiqiang Chen, Sameena Kumar, Kexun Liu, Michele Gao, Yiren Jiang, et al. SWE-agent: Agent-computer interface for software engineering, 2024b.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Yulia Tsvetkov, Quoc Le, et al. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

- Chaoyun Zhang, Liqun Li, Shilin He, Xu Zhang, Bo Qiao, Si Qin, Minghua Ma, Yu Kang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. Ufo: A ui-focused agent for windows os interaction, 2024a. URL <https://arxiv.org/abs/2402.07939>.
- Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 1592–1604, 2024b.
- Zelin Zhao, Zhaogui Xu, Jialong Zhu, Peng Di, Yuan Yao, and Xiaoxing Ma. The right prompts for the job: Repair code-review defects with large language model, 2023. URL <https://arxiv.org/abs/2312.17485>.
- Lecheng Zheng, Zhengzhang Chen, Jingrui He, and Haifeng Chen. Mulan: multi-modal causal structure learning and root cause analysis for microservice systems. In *Proceedings of the ACM Web Conference 2024*, pp. 4107–4116, 2024.

A Contributions and Acknowledgments

Ang Zhou	Peng Di
Bingchang Liu	Peng Tang
Changle Zhang	Qi Zhang
Cheng Zeng	Qiao Su
Faolang Chen	Qing Ouyang
Fei Gao	Qingfeng Li
Feng Shi	Sheng Gao
Ganglin Wei	Shenglong Jing
Guilin Li	Silin Hu
Hang Yu	Songshan Luo
Hao Jiang	Tingting Li
Hao Liu	Wenfei Li
Hongchao Le	Wenhui Shi
Hongjun Yang	Xiao Bai
Jian Mou	Yali Wang
Jianfei Zhang	Yaodong Li
Jie Bao	Yihui He
Jie Yang	Yin Hu
Jingwei Qu	Yujing Zhang
Junwei Guo	Yun Wang
Keting Chen	Yunfeng Chen
Linxia Zhong	Zheng Li
Mao Ren	Zhitao Shen
Min Shen	Zilong Hou
Lei Lei	